

SQL LAB

Lab 28 – 36

Unit 7 · Stored Procedures, Triggers & Indexing Lab Sheet

MASTER CODE — RUN ONCE BEFORE STARTING LAB 28 TO LAB 36

```

CREATE DATABASE lab_unit7;
USE lab_unit7;

-- Table: Department
-----
CREATE TABLE Department (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(50) NOT NULL UNIQUE,
    floor VARCHAR(20) DEFAULT 'First'
);

-- Table: Employee
-----
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    address VARCHAR(100) DEFAULT 'Kathmandu',
    salary DECIMAL(10,2) CHECK (salary > 0),
    gender ENUM('Male','Female','Other'),
    dob DATE,
    is_active BOOLEAN DEFAULT TRUE,
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- Table: Employee_Log (for INSERT and DELETE triggers)
-----
CREATE TABLE Employee_Log (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_id INT,
    action VARCHAR(20),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table: Salary_Audit (for UPDATE triggers)
-----
CREATE TABLE Salary_Audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Sample Data
-----
INSERT INTO Department (dept_name, floor) VALUES
('Finance', 'First'),
('HR', 'Second'),
('IT', 'Third');

INSERT INTO Employee (name, email, address, salary, gender, dob, dept_id) VALUES
('A', 'a@example.com', 'Kathmandu', 50000, 'Male', '1995-05-10', 1),
('B', 'b@example.com', 'Pokhara', 60000, 'Female', '1994-08-15', 2),
('C', 'c@example.com', 'Birtamode', 70000, 'Male', '1990-12-01', 3),
('D', 'd@example.com', 'Kathmandu', 45000, 'Female', '1998-03-20', 1);

```

LAB EXERCISES

LAB 28: Stored Procedure — Basic Creation and Selection

Write SQL queries to perform the following stored procedure operations on the Employee table:

- 1) Create a stored procedure named SelectRecords that selects all records from the Employee table.
- 2) Create a stored procedure named MyEmployees that selects employees records of a particular address.
- 3) Create a stored procedure named MyEmployees1 that selects employees records of a particular address and department id.
- 4) Execute SelectRecords.
- 5) Execute MyEmployees with address 'Kathmandu'.
- 6) Execute MyEmployees1 with address 'Kathmandu' and dept_id = 1.
- 7) Drop the stored procedure SelectRecords.
- 8) Display all stored procedures in the current database using SHOW PROCEDURE STATUS.
- 9) Display the complete definition of MyEmployees using SHOW CREATE PROCEDURE MyEmployees.

LAB 29: Stored Procedure — Insert, Update, Delete

Write SQL queries to perform the following operations using stored procedures on the Employee table:

- 1) Write SQL query to insert records in the Employee table using a stored procedure.
- 2) Write SQL query to delete record of employee whose employee id is given in parameter using a stored procedure.
- 3) Write SQL query to update name and address of employee on the basis of salary. (Here, name, address and salary is given in parameter)
- 4) Write SQL query to update name and salary of employee on the basis of employee id and salary. (Here, name, employee id and salary is given in parameter)
- 5) Execute each procedure and verify the changes in the Employee table using SELECT.

LAB 30: Stored Procedure — OUT Parameters and Returning Data

Write SQL queries to perform the following operations using OUT parameters and stored functions:

- 1) Write SQL query to display maximum salary returned by stored procedure using an OUT parameter.
- 2) Write SQL query to display average salary of employees returned by stored procedure using an OUT parameter.
- 3) Create a procedure CountEmployeesByDept with IN p_dept_id and OUT p_count to return the number of employees in that department.
- 4) Call the procedure and display the output using a user variable (e.g., @count).
- 5) Create a procedure GetMaxSalary with OUT p_max_salary to return the maximum salary.
- 6) Create a stored function GetEmployeeCount() that returns the total number of employees using the RETURN statement.
- 7) Create a stored function GetAverageSalary(p_dept_id INT) that returns the average salary of a department.
- 8) Explain the difference between OUT parameter in a procedure and RETURN statement in a function.

LAB 31: Triggers — INSERT (BEFORE and AFTER)

Using the Employee and Employee_Log tables, write SQL queries to perform the following trigger operations:

- 1) Create a BEFORE INSERT trigger named trg_before_insert_employee on Employee that sets a default email if email is NULL.
- 2) Insert a new employee with email = NULL and verify that the trigger set the default email.
- 3) Create an AFTER INSERT trigger named trg_after_insert_employee on Employee that inserts a log row into Employee_Log.
- 4) Insert a new employee and verify that the log table is updated.
- 5) Show all triggers in the database using SHOW TRIGGERS.
- 6) Drop the trigger trg_before_insert_employee using DROP TRIGGER.

LAB 32: Triggers — UPDATE (BEFORE and AFTER)

Using the Employee and Salary_Audit tables, write SQL queries to perform the following trigger operations:

- 1) Create a BEFORE UPDATE trigger named trg_before_update_salary on Employee that prevents salary from being decreased.
- 2) Try to update a salary to a lower value. Observe the error.
- 3) Create an AFTER UPDATE trigger named trg_after_update_salary on Employee that logs old and new salary into Salary_Audit.
- 4) Update a salary and verify that Salary_Audit is updated.
- 5) Update multiple employees' salaries and verify that multiple rows are logged.
- 6) Show all triggers again.

LAB 33: Triggers — DELETE (BEFORE and AFTER)

Using the Employee and Employee_Log tables, write SQL queries to perform the following trigger operations:

- 1) Create a BEFORE DELETE trigger named trg_before_delete_employee on Employee that prevents deletion of employees with salary greater than 100000.
- 2) Try to delete an employee with salary greater than 100000. Observe the error.
- 3) Create an AFTER DELETE trigger named trg_after_delete_employee on Employee that logs the deleted employee into Employee_Log.
- 4) Delete an employee with salary less than 100000 and verify that Employee_Log is updated.
- 5) Drop the trigger trg_before_delete_employee.
- 6) Show all remaining triggers.

LAB 34: Triggers — Multi-Row and Dropping

Using the Employee and Salary_Audit tables, write SQL queries to perform the following operations:

- 1) Create an AFTER UPDATE trigger trg_salary_audit_multi on Employee that logs every row whose salary is updated.
- 2) Run a multi-row update:

```
UPDATE Employee SET salary = salary * 1.10 WHERE dept_id = 1;
```
- 3) Show Salary_Audit and verify that multiple rows were logged.
- 4) Drop the trigger trg_salary_audit_multi.
- 5) Show all triggers in the database after dropping.

LAB 35: Indexing — Create, Show, Drop, Types

Using the Employee table, write SQL queries to perform the following indexing operations:

- 1) Create an index named `idx_employee_name` on `Employee(name)`.
- 2) Create a composite index named `idx_employee_address_salary` on `Employee(address, salary)`.
- 3) Create a unique index on `Employee(email)`.
- 4) Create a fulltext index named `idx_employee_address_fulltext` on `Employee(address)`.
- 5) Show indexes from `Employee` using `SHOW INDEX FROM Employee`.
- 6) Drop `idx_employee_name`.
- 7) Create index `idx_dept_salary` on `Employee(dept_id, salary)`.
- 8) Drop `idx_dept_salary`.
- 9) Show all indexes again.

LAB 36: Indexing — Multiple Key Access and Optimization

Using the Employee table, write SQL queries to perform the following indexing and optimization operations:

- 1) Create index `idx_dept_salary` on `Employee(dept_id, salary)`.
- 2) Write and test the following queries:

```
SELECT * FROM Employee WHERE dept_id = 1;
SELECT * FROM Employee WHERE dept_id = 1 AND salary > 50000;
SELECT * FROM Employee WHERE salary > 50000;
```

- 3) Use `EXPLAIN` for each query. Which query uses the index? Which does not?
- 4) Drop the index.
- 5) Use `EXPLAIN` again for the same queries. Observe the difference.
- 6) Explain the leftmost prefix rule in composite indexes.